

Session 13: Wrapper Classes and the Power of Strings

This session delves into two fundamental concepts in Java: Wrapper Classes, which bridge the gap between primitive types and the object-oriented world, and the versatile String class, along with its mutable counterparts, StringBuffer and StringBuilder.

Lecture:

Wrapper Classes and Constant Pools

In Java, we have eight primitive data types (byte, short, int, long, float, double, char, boolean). These are not objects and live on the stack, offering high performance. However, many Java frameworks and the entire Collection API are built to work with objects, not primitives.

To solve this, Java provides **Wrapper Classes**. Each primitive type has a corresponding wrapper class in the java.lang package:

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

These classes "wrap" a primitive value in an object, allowing it to be used where an object is required (e.g., in ArrayList<Integer>).

Autoboxing and Unboxing:

To make working with wrapper classes seamless, Java introduced autoboxing and unboxing in

Java 5.

- **Autoboxing:** The automatic conversion of a primitive type to its corresponding wrapper class object.
- **Unboxing:** The automatic conversion of a wrapper class object back to its corresponding primitive type.

Example:

Java

```
// Autoboxing: The primitive int '10' is automatically converted to an Integer object.
```

```
Integer integerObject = 10;
```

```
// Unboxing: The Integer object 'integerObject' is automatically converted to a primitive int.
```

```
int primitiveInt = integerObject;
```

Wrapper Class Constant Pools:

To save memory and improve performance, Java maintains a "pool" of immutable wrapper class objects for certain ranges of values. When a wrapper object is created within this range, instead of creating a new object, the JVM returns an existing object from the pool.

- **Integer Pool:** By default, Integer objects for values from **-128 to 127** are cached.
- **Other Integral Types:** Byte, Short, and Long also have pools for the range -128 to 127.
- **Character Pool:** Caches characters from \u0000 to \u007F (0 to 127).
- **Boolean Pool:** Caches Boolean.TRUE and Boolean.FALSE.

Floating-point types (Float and Double) do not have a constant pool.

Example Demonstrating the Integer Pool:

Java

```
public class IntegerPoolDemo {  
    public static void main(String[] args) {  
        // i1 and i2 will point to the SAME object from the pool.  
        Integer i1 = 100;  
        Integer i2 = 100;
```

```

System.out.println("i1 == i2 (within pool range): " + (i1 == i2)); // true

// i3 and i4 are outside the -128 to 127 range.
// A new object will be created for each.
Integer i3 = 200;
Integer i4 = 200;
System.out.println("i3 == i4 (outside pool range): " + (i3 == i4)); // false

// Always use .equals() for value comparison with objects!
System.out.println("i3.equals(i4): " + i3.equals(i4)); // true
}
}

```

The String Class, StringBuffer, and StringBuilder

The String Class and the String Pool:

In Java, String is an object that represents a sequence of characters. The most important characteristic of the String class is its **immutability**. Once a String object is created, its value cannot be changed. Any operation that appears to modify a string (like concatenation) actually creates a new String object.

To optimize memory usage due to this immutability, the JVM maintains a special area in the heap called the **String Constant Pool**.

- When you create a string literal (e.g., String s = "hello";), the JVM checks the string pool.
- If the string "hello" already exists in the pool, the reference s will point to the existing object.
- If it doesn't exist, a new String object is created in the pool, and s will point to it.
- When you use the new keyword (e.g., String s = new String("hello");), you explicitly force the creation of a new String object on the heap, outside the pool.

Example:

Java

```

public class StringPoolDemo {
    public static void main(String[] args) {
        String s1 = "Java"; // Goes into the String Constant Pool
        String s2 = "Java"; // Points to the same object as s1 from the pool
    }
}

```

```

System.out.println("s1 == s2 (from pool): " + (s1 == s2)); // true

String s3 = new String("Java"); // Created on the heap, outside the pool
System.out.println("s1 == s3 (pool vs heap): " + (s1 == s3)); // false

// .equals() always checks the actual character sequence
System.out.println("s1.equals(s3): " + s1.equals(s3)); // true
}
}

```

StringBuffer and StringBuilder:

When you need to perform a lot of string modifications (concatenation, insertion, deletion), using the String class is inefficient because it creates a new object for every change. For such scenarios, Java provides two mutable alternatives: StringBuffer and StringBuilder.

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Thread-Safety	Thread-safe (by nature)	Thread-safe (synchronized)	Not thread-safe (faster)
Performance	Slow for modifications	Slower (due to sync)	Faster (no sync overhead)

- **StringBuilder:** The recommended choice for most use cases, especially in a single-threaded environment, due to its superior performance.
- **StringBuffer:** Use this when you need a mutable string that will be accessed and modified by multiple threads simultaneously.

Example:

Java

```

public class MutableStringsDemo {
    public static void main(String[] args) {
        // StringBuilder is generally preferred
        StringBuilder sb = new StringBuilder("Hello");
    }
}

```

```
// Modifications do not create new objects
sb.append(" World");
sb.insert(5, ", Cruel");
sb.reverse();

System.out.println(sb.toString()); // dlroW leuR ,olleH

// StringBuffer has the same methods but is thread-safe
StringBuffer sbf = new StringBuffer("Test");
sbf.append("ing");
System.out.println(sbf); // Testing
}
```

Lab 12 & 13: Hands-On Practice

Lab 12: Wrapper Classes in Action

Objective: Understand boxing, unboxing, and the utility methods of wrapper classes.

Task 1: Create a sample class to understand boxing & unboxing.

Java

```
// File: BoxingDemo.java
public class BoxingDemo {
    public static void main(String[] args) {
        // 1. Autoboxing: primitive int to Integer object
        int primitiveInt = 50;
        Integer wrapperInt = primitiveInt; // Autoboxing
        System.out.println("Primitive int: " + primitiveInt);
        System.out.println("Autoboxed Integer: " + wrapperInt);

        // 2. Unboxing: Integer object to primitive int
        Integer anotherWrapper = new Integer(120);
        int anotherPrimitive = anotherWrapper; // Unboxing
        System.out.println("\nAnother Integer object: " + anotherWrapper);
        System.out.println("Unboxed primitive int: " + anotherPrimitive);
    }
}
```

```
// 3. Boxing in action with a collection
java.util.ArrayList<Integer> numberList = new java.util.ArrayList<>();
// Autoboxing happens here as we add primitives to a list of Objects
numberList.add(25);
numberList.add(35);

// Unboxing happens when we retrieve and use the value
int sum = numberList.get(0) + numberList.get(1);
System.out.println("\nSum of numbers in list: " + sum);
}
}
```

Task 2: Use different methods of Java-defined wrapper classes.

Java

```
// File: WrapperMethodsDemo.java
public class WrapperMethodsDemo {
    public static void main(String[] args) {
        // --- Integer Methods ---
        String numStr = "123";
        int parsedInt = Integer.parseInt(numStr); // Converts String to primitive int
        System.out.println("Parsed int: " + (parsedInt + 7));

        Integer intVal = Integer.valueOf("250"); // Returns an Integer object
        System.out.println("Integer valueOf: " + intVal);
        System.out.println("Binary string of 250: " + Integer.toBinaryString(intVal));

        // --- Double Methods ---
        Double d1 = 3.14;
        Double d2 = 2.71;
        System.out.println("\nComparing Doubles: " + d1.compareTo(d2)); // Returns > 0 if d1 > d2
        System.out.println("Is d1 NaN? " + Double.isNaN(d1));
        System.out.println("d1 as int: " + d1.intValue());

        // --- Character Methods ---
        char ch1 = 'A';
        char ch2 = '7';
        char ch3 = ' ';
```

```

    System.out.println("\nIs " + ch1 + " a letter? " + Character.isLetter(ch1));
    System.out.println("Is " + ch2 + " a digit? " + Character.isDigit(ch2));
    System.out.println("Is " + ch3 + " whitespace? " + Character.isWhitespace(ch3));
    System.out.println("Uppercase of 'a': " + Character.toUpperCase('a'));
}
}

```

Lab 13: String Manipulation

Objective: Get comfortable with creating and manipulating strings using String, StringBuffer, and StringBuilder.

Task: Create StringDemo class and perform different string manipulation methods.

Java

```

// File: StringDemo.java
public class StringDemo {
    public static void main(String[] args) {
        // --- Using the immutable String class ---
        System.out.println("--- String Class ---");
        String original = " Java is Fun! ";
        System.out.println("Original String: " + original + "");
        System.out.println("Length: " + original.length());
        System.out.println("Trimmed: " + original.trim() + "");
        System.out.println("Uppercase: " + original.toUpperCase());
        System.out.println("Substring (from index 8): " + original.trim().substring(8) + "");
        System.out.println("Replace 'Fun' with 'Awesome': " + original.trim().replace("Fun",
"Awsome"));
        System.out.println("Starts with ' Java'? " + original.startsWith(" Java"));

        // Splitting a string
        String sentence = "Learning Java is a great experience";
        String[] words = sentence.split(" ");
        System.out.println("Words in the sentence:");
        for (String word : words) {
            System.out.println("- " + word);
        }

        // --- Using the mutable StringBuilder class ---
    }
}

```

```
System.out.println("\n--- StringBuilder Class ---");
StringBuilder sb = new StringBuilder("Start");
System.out.println("Initial StringBuilder: " + sb);

// Append content
sb.append(" and ");
sb.append("Finish.");
System.out.println("After append: " + sb);

// Insert content
sb.insert(6, "Middle ");
System.out.println("After insert: " + sb);

// Delete content
sb.delete(6, 13); // Deletes "Middle "
System.out.println("After delete: " + sb);

// Reverse the string
sb.reverse();
System.out.println("After reverse: " + sb);
}
```